Laboratorio No.1 – Semana 16 - Swagger y Health Check para WebStorageSample

Nombre: _____ **Carné**: _____

Objetivo: Configurar Swagger para documentar endpoints HTTP.

- Crear GET /health para comprobar que la aplicación responde.
- Crear GET /api/info para publicar información general de la aplicación.
- Probar los endpoints desde Swagger y verificar sus respuestas HTTP/JSON.
- Mantener funcionando la página Razor de carga de imágenes y registrar los cambios en Git.

Repositorio
Utilice como punto de partida el repositorio de actividad 1 semana 14
Basado en GitHub: https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet

Actividad

Paso 1: revisar el paquete y Startup.cs

Desde la carpeta donde está el archivo .csproj, verifica el paquete:

```
dotnet list package
```

Si Swashbuckle.AspNetCore no aparece instalado, agrega el paquete con:

```
dotnet add package Swashbuckle.AspNetCore
```

En Startup.cs, el método ConfigureServices debe incluir los servicios siguientes. Conserva cualquier otro servicio que ya necesite el proyecto:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddRazorPages();
    services.AddEndpointsApiExplorer();
    services.AddSwaggerGen();
}
```

No dupliques AddRazorPages() ni AddSwaggerGen() si ya existen. AddEndpointsApiExplorer() ayuda a descubrir endpoints definidos con MapGet

---

Paso 2: habilitar Swagger

Dentro de Configure(), habilita Swagger en desarrollo. Coloca este bloque después del manejo de errores y antes de UseHttpsRedirection():

```
if (env.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

La interfaz se abrirá manualmente en /swagger. No es necesario configurar el navegador para que Swagger se abra automáticamente.

Paso 3: crear los endpoints

Localiza el bloque app.UseEndpoints(...) al final de Configure(). Conserva endpoints.MapRazorPages() y agrega los dos endpoints dentro del mismo bloque:

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapRazorPages();

    endpoints.MapGet("/health", () =>
        Results.Ok(new
        {
            status = "Healthy",
            timestamp = DateTime.UtcNow
        }))
        .WithName("GetHealth")
        .WithTags("Health")
        .WithSummary("Verifica que la aplicación responde");

    endpoints.MapGet("/api/info", () =>
        Results.Ok(new
        {
            application = "WebStorageSample",
            description = "Aplicación de ejemplo para Azure Blob Storage",
            version = "1.0.0"
        }))
        .WithName("GetApplicationInfo")
        .WithTags("Application")
        .WithSummary("Obtiene información general de la aplicación");
});
```

Si el compilador no reconoce Results, agrega using Microsoft.AspNetCore.Http; al inicio de Startup.cs.
No elimines el resto de las instrucciones de Configure().

Paso 4: compilar y ejecutar

```
dotnet clean
dotnet restore
dotnet build
dotnet run
```

Si la compilación falla, corrige los errores antes de continuar. Usa el puerto HTTPS que indique la terminal; en el ejemplo de clase se utilizó https://localhost:5001.

Paso 5: probar desde Swagger

Abre en el navegador: https://localhost:5001/swagger
Debes encontrar los grupos Health y Application. Ejecuta cada operación con Try it out y Execute.

5.1 GET /health

Respuesta esperada: HTTP 200 OK y un JSON similar a este:

```
{
  "status": "Healthy",
  "timestamp": "2026-10-..."
}
```

5.2 GET /api/info

Respuesta esperada: HTTP 200 OK y un JSON similar a este:

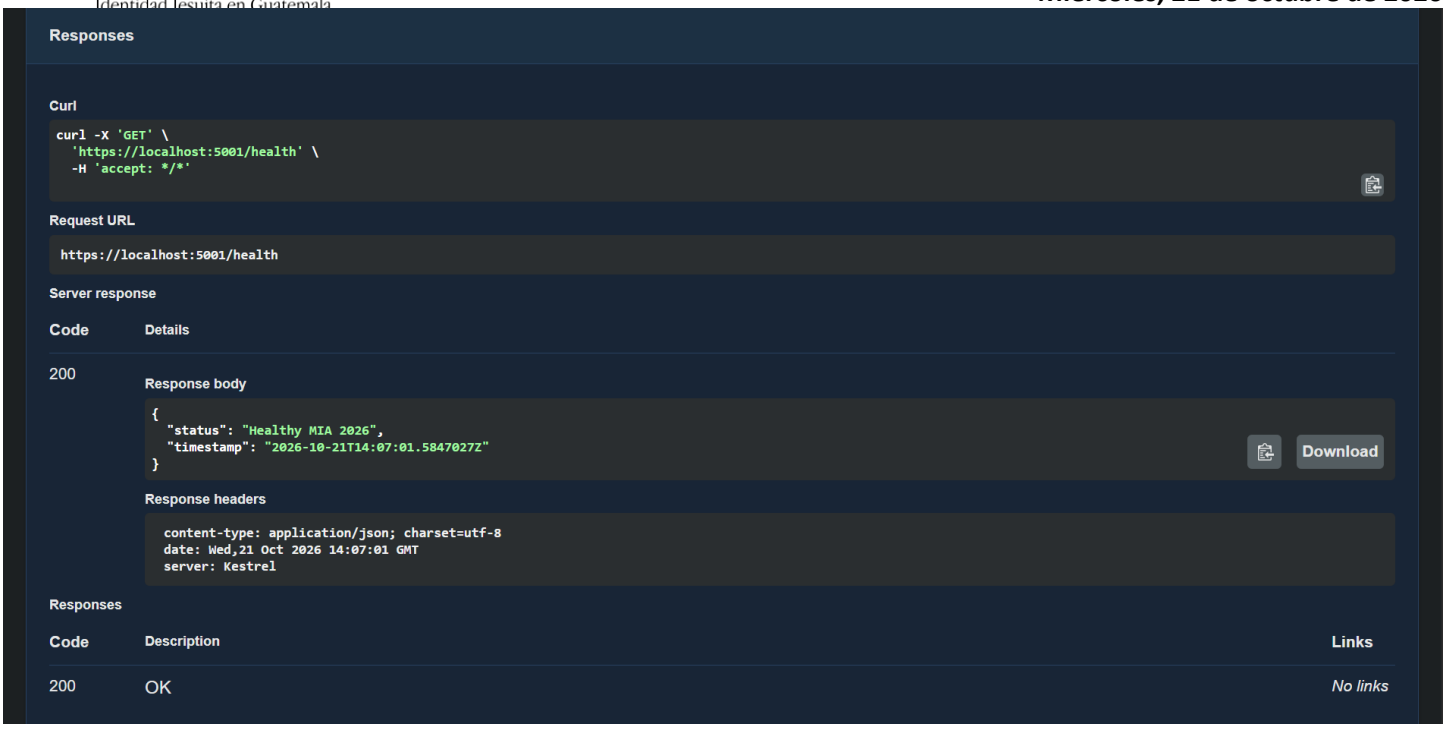
```
{
  "application": "WebStorageSample",
  "description": "Aplicación de ejemplo para Azure Blob Storage",
  "version": "1.0.0"
}
```

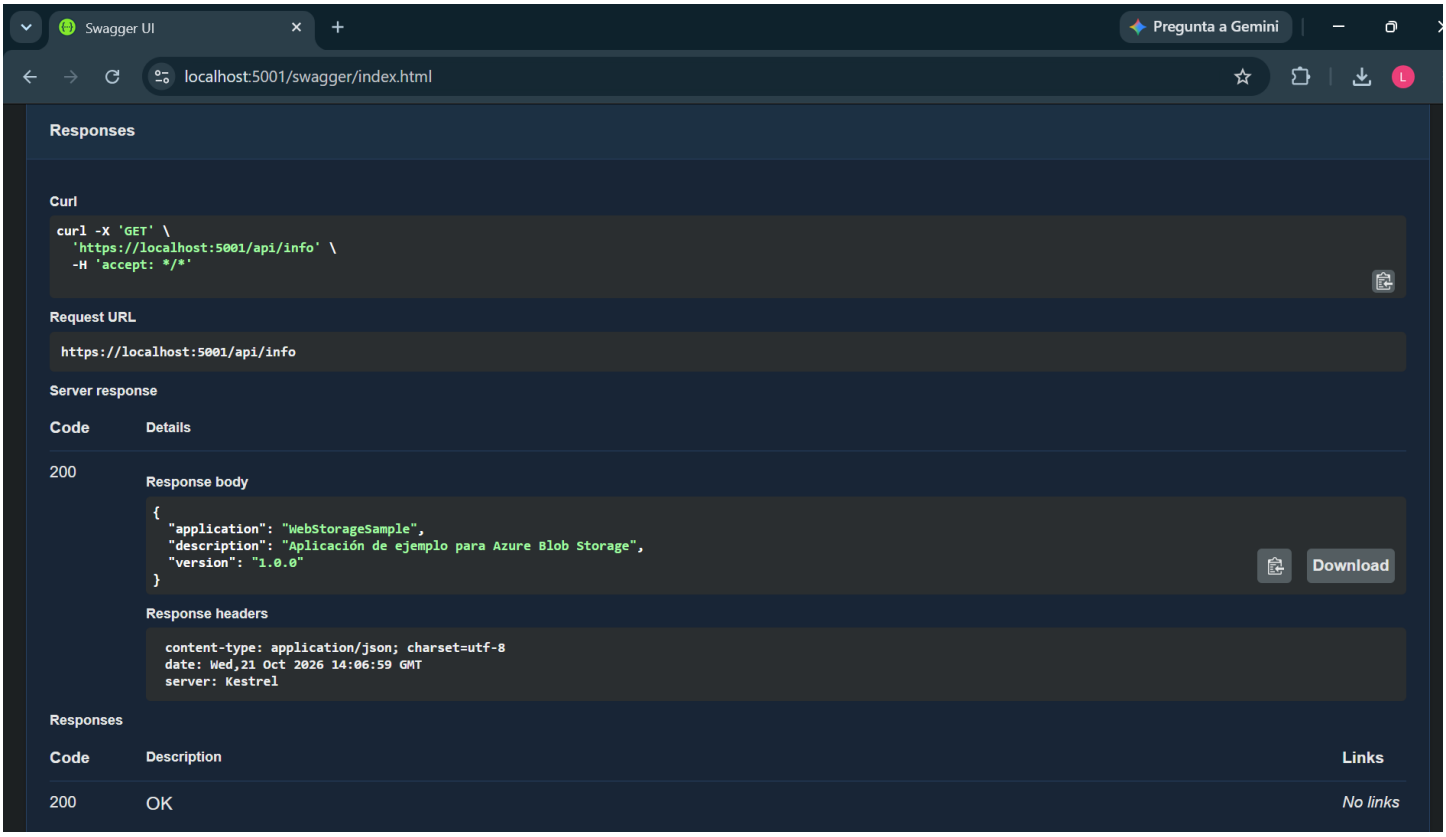
Preguntas de análisis

1. ¿Por qué Swagger mostraba “No operations defined in spec!” antes de crear los endpoints?
   Solo había Razor Pages, que no cuentan como operaciones de API. El swagger.json salía vacío, así que Swagger no tenía nada que mostrar.
2. ¿Qué diferencia existe entre /health y /api/info?
   /health indica si la app está viva y respondiendo. /api/info devuelve datos descriptivos: nombre, descripción y versión.
3. ¿Qué comprueba el endpoint /health y qué no comprueba?
   Comprueba que la app corre y responde 200. No comprueba dependencias externas como Azure Blob Storage, así que devuelve “Healthy” aunque Azure esté caído.
4. ¿Qué función cumplen AddSwaggerGen() y AddEndpointsApiExplorer()?
   AddEndpointsApiExplorer() expone los metadatos de los endpoints (ruta, método, respuestas). AddSwaggerGen() usa esos metadatos para generar el swagger.json.
5. ¿Qué sería necesario implementar para comprobar también el acceso real a Azure Blob Storage?
   Instalar Azure.Storage.Blobs, registrar un BlobServiceClient con la cadena de conexión y hacer que /health consulte el contenedor (por ejemplo con ExistsAsync()). Si funciona devuelve 200, y si falla, 503. Lo ideal es usar AddHealthChecks() con IHealthCheck.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-16/lab1/ y coloque en ella un PDF con la solución del laboratorio.
Enviar link de github a plataforma de la universidad.


Responses
Curl
curl -X 'GET' \
'https://localhost:5001/health' \
-H 'accept: */*'
Request URL
https://localhost:5001/health
Server response
Code
Details
200
Response body
{
"status": "Healthy MIA 2026",
"timestamp": "2026-10-21T14:07:01.5847027Z"
}
Download
Response headers
content-type: application/json; charset=utf-8
date: Wed,21 Oct 2026 14:07:01 GMT
server: Kestrel
Responses
Code
Description
Links
200
OK
No links


Swagger UI
Pregunta a Gemini
localhost:5001/swagger/index.html
Responses
Curl
curl -X 'GET' \
'https://localhost:5001/api/info' \
-H 'accept: */*'
Request URL
https://localhost:5001/api/info
Server response
Code
Details
200
Response body
{
"application": "WebStorageSample",
"description": "Aplicación de ejemplo para Azure Blob Storage",
"version": "1.0.0"
}
Download
Response headers
content-type: application/json; charset=utf-8
date: Wed,21 Oct 2026 14:06:59 GMT
server: Kestrel
Responses
Code
Description
Links
200
OK
No links